

Laborator 7

Implementarea sistemelor in timp real. Comunicarea între task-uri

Obiectiv În acest laborator se realizează implementarea tehnologiilor prezentate în laboratoarele anterioare privitoare la sistemele în timp real: descrierea unei aplicații sub formă de task-uri, comutarea task-urilor, administrarea task-urilor pe bază de priorități și alocarea resurselor pentru un task. Implementarea se va realiza pe machetele din laborator C8051FX20 cu microcontrolere C8051F120. De asemenea se va concepe un sistem în timp real realizat cu task-uri care comunică între ele folosind tehnica transmisiei de mesaje pe canale de comunicație.

Breviar teoretic

1. Prezentare macheta C8051FX20.

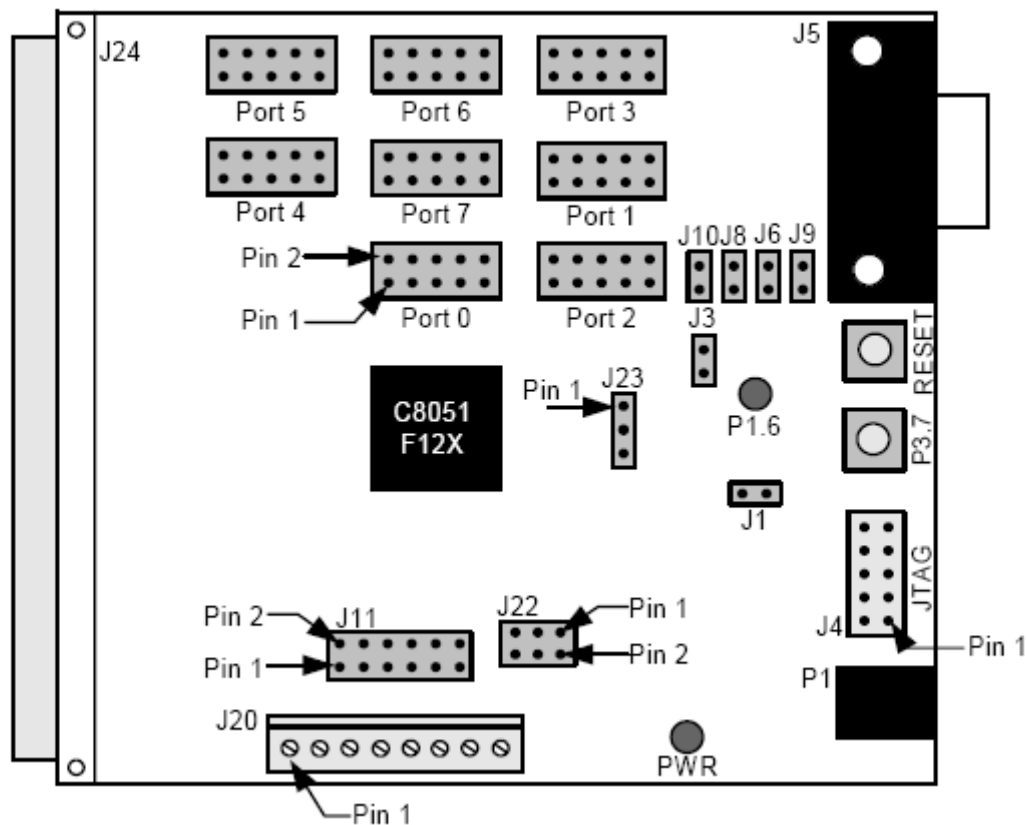


Fig.1. Schema bloc macheta C8051FX20

Conector machetă	Denumire
P1	Conector alimentare (7-15V)
J1	Conectează butonul SW2 la pinul P3.7
J3	Conectează LED D3 la pinul P1.6

J4	Conector JTAG pentru testare și programare machetă
J5	Conector DB9 (9 pini) pentru portul UART0 RS232
J6	Conector pentru pinul UART0 Tx (P0.0)
J8	Conector pentru pinul RTS (P4.0)
J9	Conector pentru pinul RX (P0.1)
J10	Conector pentru pinul CTS (P4.1)
J11	Conector buclă analogică
J12 ... J19	Conectori porturi paralele P0 ... P7
J20	Terminal intrare-ieșire analogic
J22	Conector tensiune referință (pentru convertor AD)
J23	Dezactivare monitor tensiune alimentare
J24	Conector expansiune pe 96 pini

Cele două butoane și LED-urile sunt conectate conform tabelului de mai jos:

Descriere	I/O	Legătură
SW1	Reset	Direct
SW2	P3.7	J1
LED verde	P1.6	J3
LED roșu	Alimentare	Direct

Conectorul JTAG (J4) are următoarea distribuție a pinilor:

Pin	Descriere
1	Alimentare (3.3V)
2, 3, 9	GND
4	TCK
5	TMS
6	TDO
7	TDI
8, 10	Neconectați

Notă: firul roșu de la panglica programator JTAG se conectează la pinul de alimentare!

Conectorul buclă analogică (J11) are următoarea distribuție a pinilor:

Pin	Descriere
1	CP0+
2	CP0-
3	DAC0
4	DAC1
5	AIN0.0
6	AIN0.1
7	VREF

8	AGND
---	------

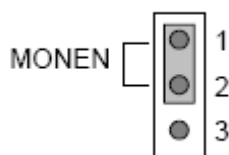
Terminalul intrare ieșire analogic (J20) are următoarea descriere a pinilor:

Pin	Descriere
1	CP0+
2	CP0-
3	DAC0
4	DAC1
5	CP1+
6	CP1-
7	AIN0.0
8	AIN0.1
9	DAC0
10	DAC1
11	AIN0.6
12	AIN0.7

Pentru fiecare conector port IO (J12 – J19) avem următoarea configurație a pinilor:

Pin	Descriere
1	Px.0
2	Px.1
3	Px.2
4	Px.3
5	Px.4
6	Px.5
7	Px.6
8	Px.7
9	Vcc (+3V)
10	GND (0V)

Conectorul dezactivare monitorizare tensiune alimentare are următoarea configurație:



2. Lucrul cu mediul de dezvoltare – testare – programare SiLabs IDE

Operarea cu microcontroler-ul C8051F120 se face prin intermediul unui mediu compact de dezvoltare – testare – programare. Similar cu IAR pentru familia AVR, și aici sunt permise o serie de operații care se pot realiza în dezvoltarea aplicației:

- crearea unui proiect și includerea de fișiere în proiect;

- compilarea codurilor sursă și crearea fișierelor obiect (compile, build);
- conectarea la machetă și descărcarea fișierului obiect în machetă;
- rularea programului, rularea pas cu pas, vizualizarea resurselor și a variabilelor.

Întrucât etapele sunt similare cu cele ale mediului IAR, prezentat în laboratoarele anterioare, aici nu vor fi prezentate decât sumar:

Crearea unui proiect și includerea de fișiere în proiect

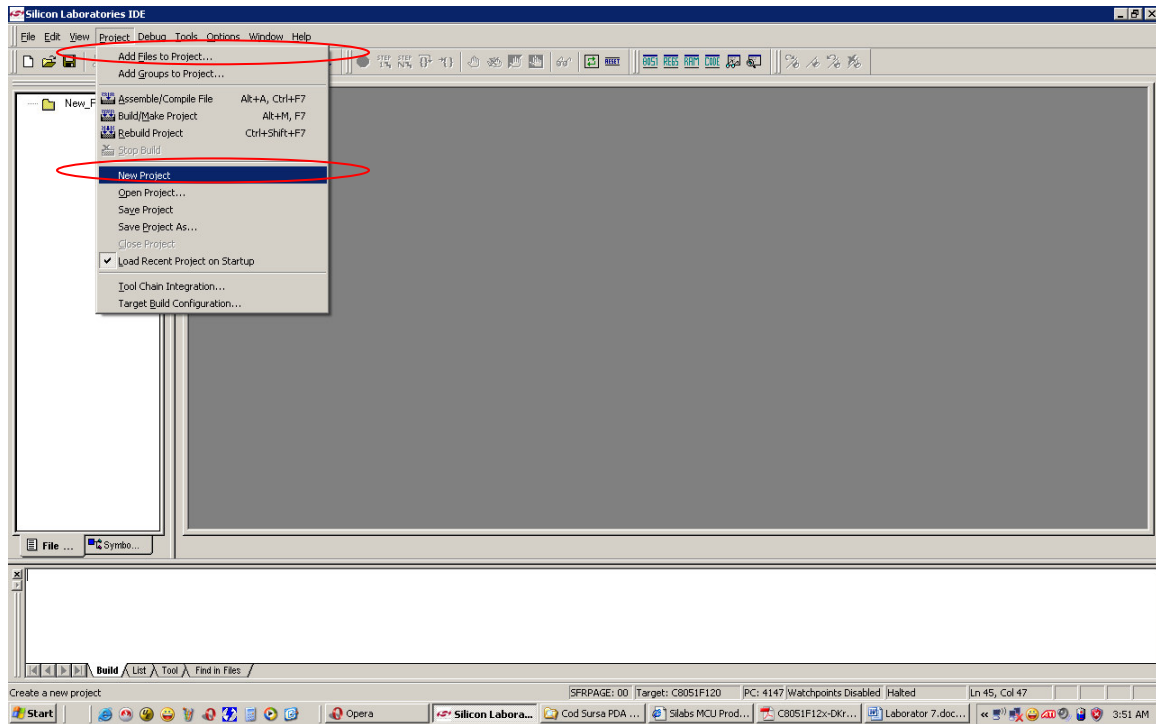


Fig. 2. Crearea unui proiect și adăugarea de fișiere în proiect

La pornirea mediului SiLabs IDE întodeauna este creat un proiect nou. Dacă se dorește crearea unui proiect nou pe parcurs se selectează din meniu Project – New Project. Adăugarea fișierelor la un proiect se face din Project – Add Files to Project. Crearea unui fișier sursă nou se face prin selectarea File – New File și apoi se salvează.

Compilarea codului sursă și construirea fișierelor obiect, conectarea la machetă și descărcarea programului în machetă, rularea programului și vizualizarea resurselor și variabilelor sistemului sunt prezentate în figura de mai jos.

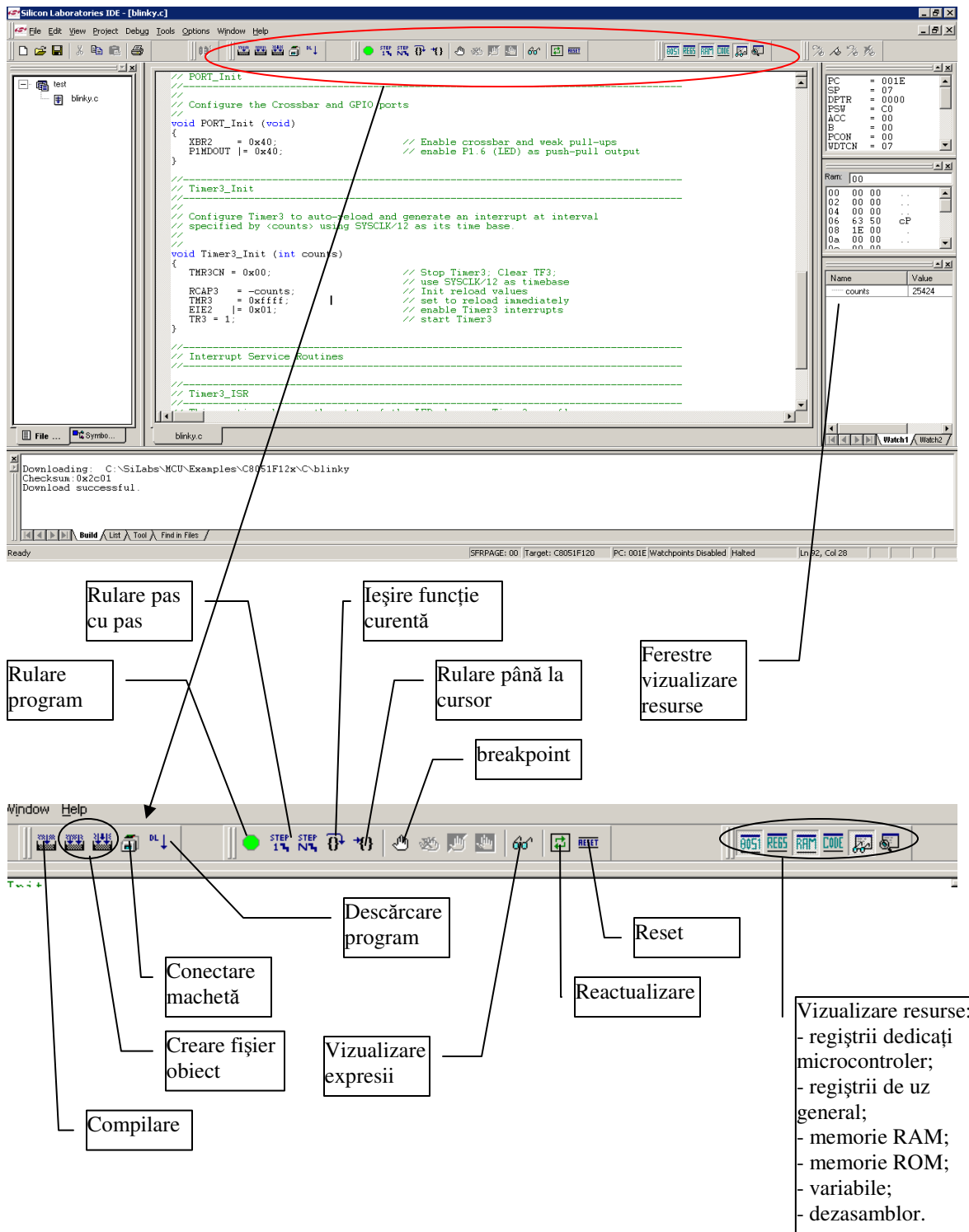


Fig.3. Sesiune de lucru cu SiLabs IDE

2. Configurări circuite periferice la C8051F120

Microcontroler-ul C8051F120 se bazează pe un nucleu de 8051. Setul intern de regiștrii ai acestuia au fost concepuți astfel încât aceștia să fie compatibili cu cei de la un microcontroler 8051 clasic. După cum a fost prezentat și la curs, microcontroler-ul C8051F120 este unul mult mai complex ca arhitectură internă ca un 8051 clasic, deci posedă un număr de circuite periferice superior față de un 8051 clasic. Pentru regiștrii suplimentari s-au introdus locații suplimentare în spațiul SFR (special function register) folosindu-se mai multe pagini SFR.

În problemele prezentate în continuare se va specifica acolo unde sunt folosiți regiștrii suplimentari și unde se vor folosi regiștrii tipici 8051.

Circuitul 8051 a fost studiat la cursul intitulat Sisteme cu microprocesoare (anul IV). La acest laborator, ca anexă, a fost atașat laboratorul 6 de la cursul sus amintit în care sunt explicații regiștrii de configurare a circuitelor periferice.

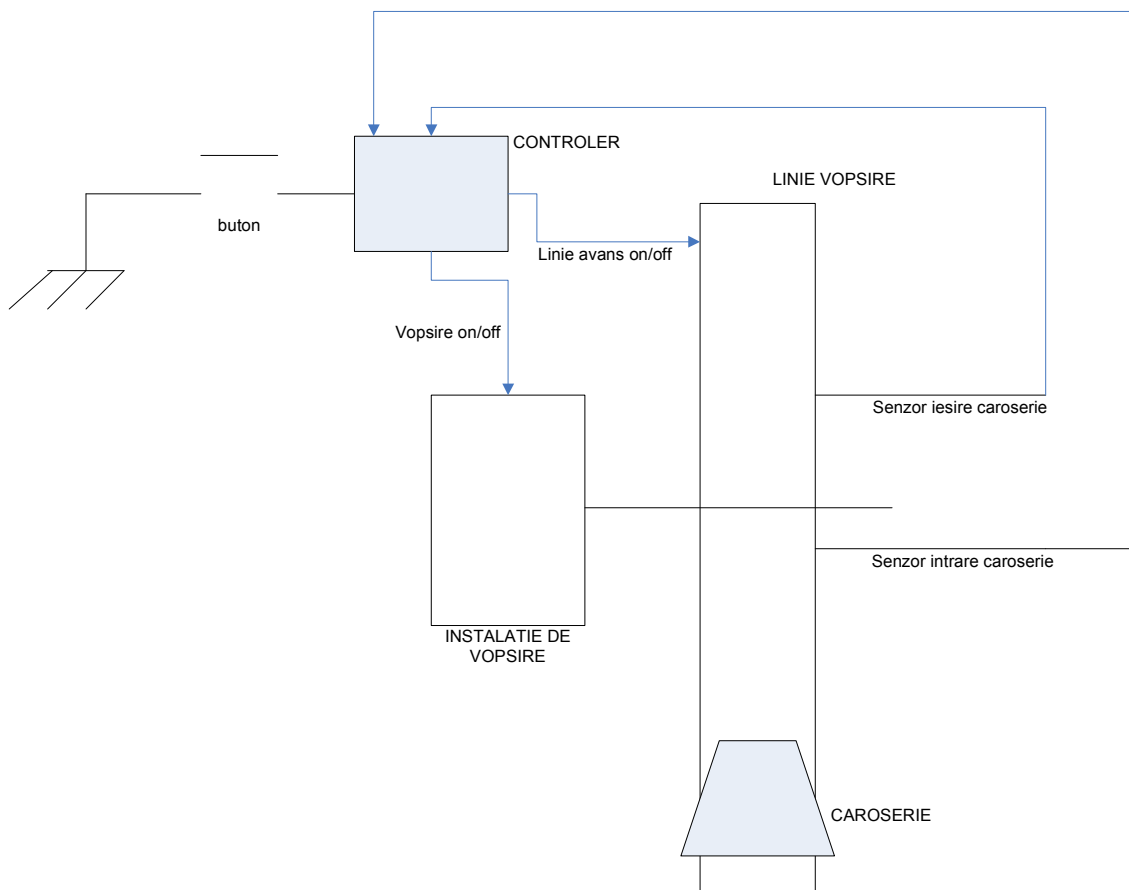
Pentru regiștrii suplimentari se vor da explicații acolo unde sunt utilizați.

3. Desfășurarea lucrării

Să se realizeze un sistem în timp real pentru comanda unui automat care realizează vopsirea caroseriilor pe linia de fabricație. Condițiile de lucru sunt următoarele:

- pe linie este pusă caroseria;
- un supraveghetor acționează un buton pentru avansarea liniei;
- la detectarea caroseriei în dreptul instalației de vopsire este pornită pompa de vopsire;
- pompa de vopsire este menținută pornită până când caroseria a trecut din dreptul instalației sau supraveghetorul acționează din nou butonul;
- la acționarea butonului se oprește instalația de vopsire și avansarea liniei pentru a permite ajustări manuale;
- dacă nu se acționează butonul pompa de vopsire se oprește când a fost detectată trecerea caroseriei de instalația de vopsire, moment în care se oprește și avansarea liniei.

Schema sistemului este următoarea:



Sistemul are următoarele intrări:

- buton – plasat pe pinul P3.7 de la portul paralel. Stări: 0 apăsăat 1 neapăsăat;
- senzor intrare caroserie – recepția serială a caracterului 1;
- senzor ieșire caroserie – recepția serială a caracterului 2.

Sistemul acționează următoarele comenzi:

- linie avans – comandă dată pe pinul P1.6. Stări: 0 – oprită , 1 – pornită;
- vopsire – comandă transmisă pe portul serial. Când este transmis mesajul „Vopsire on” – este pornit procesul de vopsire și când este transmis „Vopsire off” este oprit procesul de vopsire.

Durata maximă de răspuns admisă de la apariția unui eveniment este de maxim 500 ms. Intervalul de timp de la pornirea liniei la intrarea caroseriei în instalația de vopsire este de 10 secunde. Intervalul de timp cât caroseria este vopsită până la ieșirea ei din instalație este de 20 secunde. De la aplicarea comenzii de pornire – oprire instalație vopsire până la executarea ei trec 20 ms. De la aplicarea comenzii oprire pornire linie până la executarea ei trec 30 ms. Timpul de procesare – executare program se considera de 1ms.

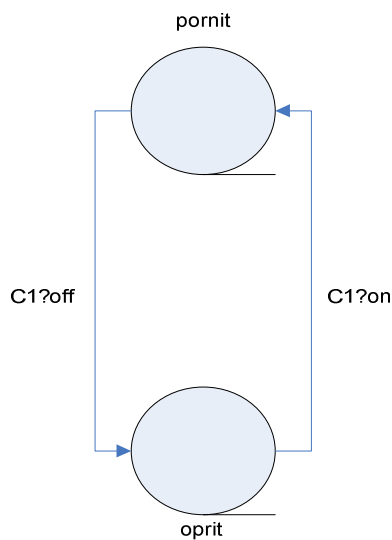
Sistemul se dorește implementat pe un microcontroler C8051F120 cu frecvența internă de lucru de 3062500MHz.

Rezolvare

A. Specificarea sistemului folosind mașini de stare extinse

S-a optat pentru o soluție a unui sistem cu trei task-uri. Task-ul 1 se ocupă de comanda liniei. Task-ul 2 se ocupă de comanda instalației de vopsire. Task-ul 3 se ocupă de preluarea stării de la senzori și buton și transmiterea comenzilor către celelalte două. Schema folosind mașini de stare extinse este următoarea:

Task-ul 1 comanda linie



Task-ul 1 comandă linie (X,V,C,L,A):

X = x: {pornit, oprit}

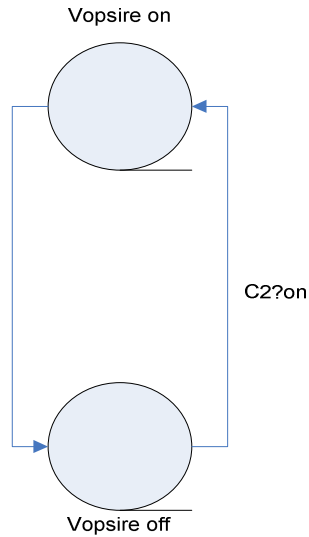
V = 0

C = c : {c1 : canal receptie comenzi de la controller}

L = off, on evenimente primite de la controller (pe canalul c1)

A = {a1,(pornit,1,c1?off,oprit)},{a2,(oprit,1,c1?on,pornit)}

Task-ul 2 comanda vopsire



Task-ul 2 comandă linie (X,V,C,L,A):

$X = x: \{\text{pornit, oprit}\}$

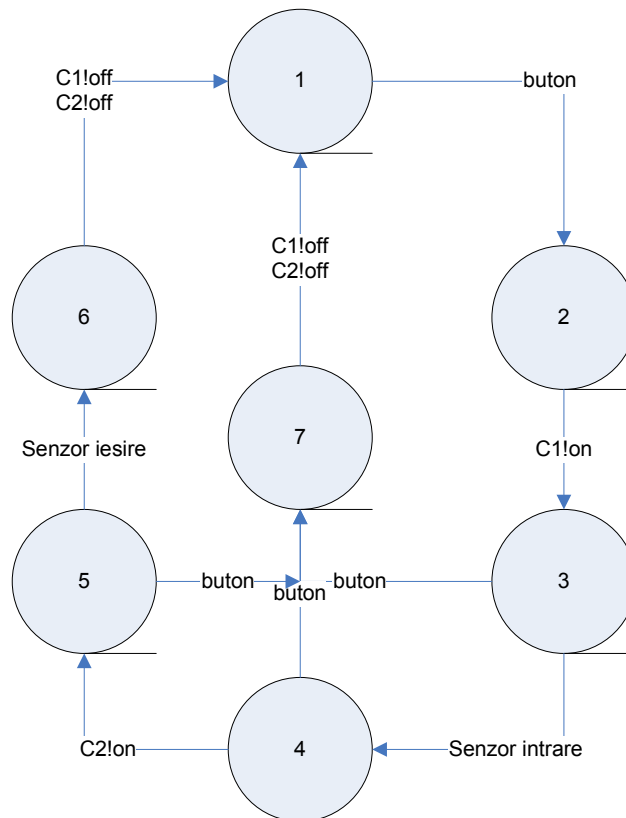
$V = 0$

$C = c: \{c2: \text{canal receptie comenzi de la controller}\}$

$L = \text{off, on}$ evenimente primite de la controller (pe canalul c1)

$A = \{a1, (\text{vopsire on}, 1, c2?off, \text{vopsire off})\}, \{a2, (\text{vopsire off}, 1, c2?on, \text{vopsire on})\}$

Task-ul 3 preluare informatii de la senzori si comandă task 1 și 2



Task-ul 3 comandă linie (X,V,C,L,A):

$X = x: \{1,2,3,4,5,6,7\};$

$V = 0;$

$C = c: \{c1: \text{canal transmitere comenzi la task-ul 1}, c2: \text{canal transmitere comenzi la task-ul 2}\};$

$L = \text{buton, senzor intrare, senzor iesire evenimente sporadice}$

$A = \{a1,(1,1,\text{buton},2)\}, \{a2,(2,1,c1!\text{on},3)\}, \{a3,(3,1,\text{senzor intrare},4)\}, \{a4,(4,1,c2!\text{on},5)\},$
 $\{a5,(5,1,\text{senzor iesire},6)\}, \{a6,(6,1,c1!\text{off} \quad c2!\text{off},1)\}, \{a7,(5,1,\text{buton},7)\},$
 $\{a8,(4,1,\text{buton},7)\}, \{a9,(3,1,\text{buton},7)\}.$

B. Planificarea și analiza fezabilității sistemului

Cerința aplicației este să asigure un timp de răspuns maxim admisibil de la recepția oricărui eveniment de 500 ms. Pe de altă parte, aplicația nu trebuie să piardă nici unul din evenimentele buton, senzor intrare sau senzor ieșire care pot apărea. Conform datelor problemei, intervalele de timp între două evenimente sunt mai mari decât intervalul de timp cerut maxim pentru un răspuns, deci crearea unei logici de răspuns într-un interval de timp mai mic de 500 ms este acoperitoare și pentru asigurarea acestora.

Intervalul de 500 ms reprezintă timpul maxim de răspuns admis de la apariția unei comenzi la răspunsul acestuia. Dacă privim ansamblul celor trei task-uri rezultă că timpul de calcul pentru task-ul 1, task-ul 2 și task-ul 3 nu trebuie să depășească cumulativ 500 ms. Pentru a se încadra în acest interval se va lua ca interval maxim admis pentru task-ul 1, 2 și 3 100ms

Conform datelor problemei timpul de calcul care l-ar necesita task-ul 1 este dat de timpul de răspuns de la aplicarea comenzii de pornire-oprire linie și de timpul de procesare adică $30\text{ms} + 1\text{ms} = 31\text{ms}$.

Pentru task-ul 2 timpul de calcul este dat de timpul de răspuns de la aplicarea comenzii de oprire – pornire vopsire și executarea ei la care se adună timpul de calcul deci $20\text{ms} + 1\text{ms} = 21\text{ms}$.

Task-ul trei fiind unul care efectuează numai procesare este de 1ms.

Se va folosi un sistem de răspuns bazat pe priorități. Task-ul care tratează evenimentele sporadice trebuie să fie cel cu gradul de prioritate cel mai mare deoarece acesta trebuie să asigure o tratare urgentă a evenimentelor când apar. Deci task-ul trei va avea $P=1$. Celalate două task-uri se vor repartiza în funcție de timpul de calcul task1 cu $P=3$ și task-ul 2 cu $P=2$

Deci se va opta pe următoarea planificare:

Task	Prioritate (P)	Timp calcul (C)	Timp admis (D)
1	3	31 ms	100 ms
2	2	21 ms	100 ms
3	1	1 ms	100 ms

Pentru analiza fezabilității se va verifica în primul rând condiția 3:

$$\forall i, \sum_{j=1}^{i-1} \frac{T_i}{T_j} C_j \leq T_i - C_i$$

Raportul T_i/T_j este 1 deci avem: $21 \text{ ms} + 1 \text{ ms} = 22 \text{ ms} < 100 \text{ ms} - 31 \text{ ms} = 69 \text{ ms}$.

Apoi se va aplica conditia suficientă de fezabilitate:

Răspunsul pentru task-ul 1 este:

$R(0,31,3) = \text{if } \text{Comp}([0,31),3) = 0 \text{ then } 31$

$\text{Else } R(31,31+\text{Comp}([0,31),3))$

$\text{Comp}([0,31),3) = I([0,31),1) \times 1 + I([0,31),2) \times 21 = ([31/100] - [0/100]) \times 1 + [31/100] - [0/100] \times 21 = 1 + 21 = 22$

Se va continua mai departe cu:

$R(31,53,3) = \text{if } \text{Comp}([31,53),3) = 0 \text{ then } 53$

$\text{Else } R(53, 53+\text{Comp}([31,53),3))$

$\text{Comp}([31,53),3) = \text{Inputs}([31,53),1) \times 1 + I([31,53),2) \times 21 = ([53/100] - [31/100]) \times 1 + ([53/100] - [31/100]) \times 21 = 0$

Deci timpul de răspuns pentru task-ul 1 (cu gradul cel mai mic de prioritate) este de 53 ms < 100 ms. Deci sistemul propus este fezabil!

C. Realizarea programului

1. Includere librării, declarare variabile și definire tipuri de date

Este secțiunea de la începutul fișierului în care se află declarate variabilele utilizate. Aici a fost definit tipul de date canal folosit pentru comunicarea între task-uri. Tot aici sunt declarate stările celor trei task-uri

```
#include <c8051f120.h> //fișier pentru declaratii
```

```
#include <stdio.h>
```

```
sbit LED = P1^6;
```

```
sbit TASTA = P3^7;
```

```
#define SYSCLK 3062500
```

```
//variabile globale folosite in program
```

```
unsigned char contor;//folosit pentru debouncing la tasta
```

```
unsigned char sem_tasta;//folosit pentru semnalizarea eveniment tasta
```

```
unsigned char tastactr;//folosit in logica de debouncing interpretare tasta
```

```
unsigned char task_crt=0;//task curent
```

```
unsigned char sem_senzor_intrare=0;//semnalizare senzor intrare 0-neactivat 1-activat
```

```
unsigned char sem_senzor_iesire=0;//semnalizare senzor iesire 0-neactivat 1-activat
```

```
unsigned char recser;//variabila pentru receptia seriala
```

```
//definire tip canal
```

```

typedef struct Canal
{
char sem_canal;//semnalizare transmitere mesaj 0-nu este mesaj 1-este mesaj
char mesaj;//corp mesaj 0-off 1-on ...
}canal;

//cele doua canale de comunicatie
canal c1,c2;

//definire stare task1, task2 si task3;

#define PORNIT 1
#define OPRIT 0

#define VOPSIRE_ON 1
#define VOPSIRE_OFF 0
#define ON 1
#define OFF 0

unsigned char stare_task1=OPRIT;
unsigned char stare_task2=VOPSIRE_OFF;
unsigned char stare_task3=1;

```

2. Funcțiile care comunică mesaje între task-uri Send și Receive

Funcțiile au fost prezentate și în cursul dedicat descrierii task-urilor folosind mașini de stare extinsă (comunicarea între task-uri).

```

//functii pentru transmisia si receptia pe canale
//transmisia unui mesaj pe un canal
void Send (canal *pc, char mesaj)
{
pc->mesaj= mesaj;
pc->sem_canal=1;
}

//receptia unui mesaj de pe un canal
unsigned char Receive(canal *pc)
{
if(pc->sem_canal)
{
pc->sem_canal = 0;
return pc->mesaj;
}
else
return -1;
}

```

}

3. Programarea microcontroler – circuite periferice

Inițializarea sistemului se face în mai multe funcții introduse în program.

Funcția `Init_System` realizează dezactivarea timer-ului circuitului watch-dog. Pentru aceasta se introduce următoarea secvență:

```
//initializare sistem
void Init_System(void)
{
    //dezactivare timer watch-dog
    WDTCN = 0xde; //succesiune de secvențe pentru dezactivare watchdog
    WDTCN = 0xad;
}
```

Funcția `Init_TMR` realizează inițializarea canalelor Timer 0 pentru a genera întreruperi la un interval de timp de 100 ms deci cu o frecvență de 10Hz, cât a fost stabilit anterior.

Pentru o frecvență de 3062500 Hz avem:

$3062500 / 12 \sim 255208 / 10 \sim 25521 = 63B0h$

$1\ 0000h - 63B0h = 9C50h$

Deci TH0 = 9Ch și TL0 = 50h

```
//initializare canale timer 0 si 1
void Init_TMR(void)
{
    SFRPAGE = TIMER01_PAGE; //pregatește pagina pentru configurare timer 0 si 1
    TCON = 0x50; //activare canal timer 0 si timer 1
    TMOD = 0x21; //temporizator canal 1 mod 2 (8 biti cu revenire), canal 0 mod 1 (16 biti fara revenire)
    CKCON = 0x10; //selectie sursa ceas timer 1 - fara prescaler 12
    TH1 = 0xfb; // - comunicare 19200bps
    TH0 = 0x9C; //divizare pentru a obtine 100 herti (10 ms) la timer 0
    TL0 = 0x50; //
}
```

Funcția `Init_Port` realizează inițializarea porturilor paralele astfel încât să poată îndeplini cerințele din problemă: să asigure comunicația serială și să aibă conectate LED și tasta.

```
//initializare porturi I/O
void Init_PORT(void)
{
    SFRPAGE = CONFIG_PAGE; //pagina pentru configurare porturi
    P0MDOUT = 0x03; //configureaza pinul TX ca iesire
}
```

```

XBR0 = 0x04; //configureaza portul UART0 pe porturile 0
//configurare port 2
XBR2 = 0x40; //configureaza liniile de port
P1MDOUT |= 0x40; //configureaza pinul P1.6 ca iesire (LED)
}

```

Funcția Init_UART0 realizează inițializarea canalului 0 al portului serial.

```

//initializare canal serial
void Init_UART0(void)
{
    SFRPAGE = UART0_PAGE; //setare pagina SFR pentru canalul 0 UART
    SCON0 = 0x50; //serial modul 1 receptie activata
}

```

Funcția enable_interrupt activează întreruperile care provin de la canalul Timer0 și de la portul serial la recepție.

```

//activare intreruperi
void enable_interrupt(void)
{
    SFRPAGE = LEGACY_PAGE;
    //activare intrerupere canal serial și
    IE = 0x92;
    //initializare pagina seriala
    //SFRPAGE = UART0_PAGE;
}

```

4. Scriere task-uri

Mai jos este prezentat codul sursa pentru task-uri descris conform mașinii de stari extinse:

```

void task1(void) //task comanda linie
{
    switch(stare_task1)
    {
        case OPRIT:
            //receptie mesaj ON canal 1?
            if(Receive(&c1) == ON)
            {
                stare_task1 = PORNIT;
                LED = 1; //linie pornita
            }
        }
    }

```

```

        printf("Linie pornita. Urmariti LED-ul!!\n");
    }
    break;
    case PORNIT:
        //receptie mesaj OFF canal 1?
        if(Receive(&c1)==OFF)
        {
            stare_task1=OPRIT;
            LED = 0;//linie oprita
            printf("Linie oprita. Urmariti LED-ul!!\n");
        }
        break;
}
}

```

```

void task2(void)//task comanda instalatie vopsea
{
    switch(stare_task2)
    {
        case VOPSIRE_OFF:
            //receptie mesaj ON canal 2?
            if(Receive(&c2)==ON)
            {
                stare_task2=VOPSIRE_ON;
                printf("Pornit vopsirea!!\n");
            }
            break;
        case VOPSIRE_ON:
            //receptie mesaj OFF pe canal 2?
            if(Receive(&c2)==OFF)
            {
                stare_task2=VOPSIRE_OFF;
                printf("Oprit vopsirea!!\n");
            }
            break;
    }
}
}

```

```

void task3(void)//task comanda controler proces
{
    switch(stare_task3)
    {
        case 1://starea 1
            if(sem_tasta)//eveniment tasta
            {
                sem_tasta=0;
            }
        }
    }
}

```

```

        stare_task3=2;
    }
    break;
    case 2://starea 2
    //trimite mesaj activare linie
    Send(&c1,ON);
    stare_task3++;
    break;
    case 3://starea 3
    if(sem_senzor_intrare)//eveniment senzor intrare
    {
        sem_senzor_intrare =0;
        stare_task3=4;
    }
    if(sem_tasta)//eveniment tasta
    {
        sem_tasta=0;
        stare_task3=6;//7;
    }
    break;
    case 4://starea 4
    //trimite mesaj activare vopsea
    Send(&c2,ON);
    stare_task3++;
    break;
    case 5://starea 5
    if(sem_senzor_iesire)//eveniment senzor iesire
    {
        sem_senzor_iesire =0;
        stare_task3 = 6;
    }
    if(sem_tasta)//eveniment tasta
    {
        sem_tasta=0;
        stare_task3=6;//7
    }
    break;
    case 6://starea 6
    //trimite mesaj inchidere vopsea si linie
    Send(&c2,OFF);
    Send(&c1,OFF);
    stare_task3 = 1;
    break;
    // case 7://starea 7 identica cu starea 6
    //trimite mesaj inchidere vopsea si linie

```

```

}
//stergere evenimente vopsire eventual generate cand linia e oprita
    sem_senzor_intrare =0;
    sem_senzor_iesire=0;

}

```

În plus la task-ul 3 s-a adăugat ștergerea evenimentelor sem_senzor în cazul în care acestea ar fi generate când linia nu este pornită.

5. Subrutine de tratare intreruperi

Este vorba de tratarea întreruperii de la timer, unde se află tratarea evenimentului de tip tasta și a administratorului de task-uri, și de întreruperea la recepție serială care tratează semnalizările senzorilor intrare și ieșire caroserie din instalația de vopsire.

```

//subrutina tratare timer
void Timer0_ISR (void) interrupt 1
{
    TH0 = 0xf6;//divizare cu 250 - comunicare 9600bps
    TL0 = 0x08;//divizare pentru a obtine 100 herti (10 ms)

    //tratare tasta pentru debouncing
    if(!tastacrt)
    {
        if(contor==0)
        {
            if(TASTA == 0)//tasta apasata
                contor = 1;//contor debouncing
        }
        else
        {
            if(TASTA == 0)//tasta apasata
            {
                tastacrt=1;
                sem_tasta=1;//semnalizare eveniment apasare tasta
            }
            contor = 0;
        }
    }
    else
    {
        if(contor==1)
        {
            if(TASTA == 1)//tasta ridicata

```

```

        contor = 1;//contor debouncing
    }
    else
    {
        if(TASTA == 1)//tasta ridicata
        {
            tastacrt = 0;
        }
        contor = 0;
    }
}
/*
contor++;
if(contor==100)
{
    LED = ~LED;
    contor = 0;
}
*/
//comutare task-uri
task_crt++;
if(task_crt==3)
    task_crt=0;
switch(task_crt)
{
    case 0: task1();
    break;
    case 1: task2();
    break;
    case 2: task3();
    break;
}
}

void Serial0_ISR (void) interrupt 4
{
    if(RI0)
    {
        recser=SBUF0;
        if((recser=='T')||(recser=='i'))
            sem_senzor_intrare=1;//semnalizare eveniment caroserie senzor intrare
        if((recser=='O')||(recser=='o'))
            sem_senzor_iesire=1;
        RI0=0;
    }
}

```

7. Programul principal

```
void main(void)
{

    Init_System();//initializare sistem
    Init_TMR();//initializare canale timer
    Init_PORT();//initializare porturi I/O
    Init_UART0();//initializare canale seriale
    enable_interrupt();//activare intreruperi


    printf("Sistem comanda linie industrială vopsire\n");
    printf("=====Evenimente=====\\n");
    printf("Tasta SW2 macheta - pornire-oprire linie\\n");
    printf("Tasta I sau i - intrare caroserie la vopsi\\n");
    printf("Tasta O sau o - iesire caroserie de la vopsi\\n");


    LED=0;
    task1();//pornire sistem operare


    for(;;)
    {
    }
}
```

Aplicații. Probleme propuse

1. Să se identifice conectorii pe machetă conform figurii 1.
2. Să se identifice pașii necesari pentru dezvoltarea și implementarea pe machetă a unei aplicații de la crearea proiectului până la rularea acestuia pe machetă.
3. Să se descrie o altă soluție de rezolvare a problemei folosind mașini de stări extinse.
4. Să se identifice arcele de acțiuni descrie folosind formalismul matematic (X,V,C,L,A) în mașina de stări extinse.
5. Să se identifice regiștrii pentru inițializarea canalelor timer și a portului serial rolul lor și modul în care sunt programați folosind laboratorul 6 de Sisteme cu microprocesoare atașat ca anexa. (TCON, TMOD, TH0, TH1, TL0, SCON)
6. Se vor introduce codurile sursă prezentate mai sus într-un singur fișier care va fi apoi compilat și implementat pe machetă.
7. Se va observa funcționarea sistemului în timp real și corespondența cu mașinile de stare extinse folosite la descriere.

8. Se cere implementarea pentru aceeași problemă dar care realizează trecerea prin două instalații de vopsire. Fiecare instalație va avea doi senzori: unul de intrare și unul de ieșire.